PIC16F877A

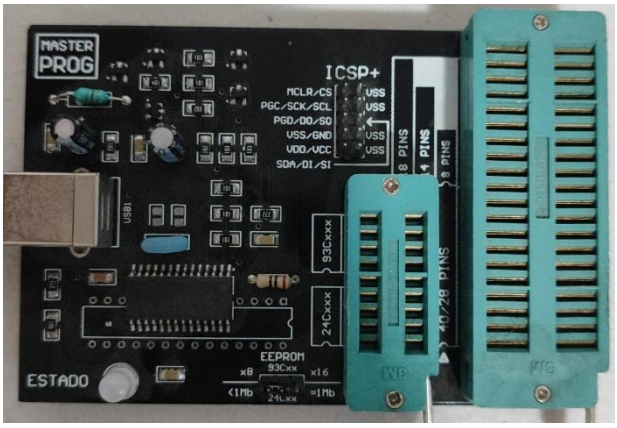
- **PIC** → familia de microcontroladores de Microchip.
- **16** → indica que pertenece a la familia **PIC16**.
- **F** → utiliza memoria de programa **Flash**.
- **877** → identifica el modelo concreto.
- **A** → revisión o versión mejorada del dispositivo.

(Microcontrolador de 8 bits)

SOFTWARE INSTALLATION

Para este caso particular se decidió instalar una versión especifica de “MPLAB X IDE” debido a lo que se tiene como “hardware”. Existe algo importante que diferenciar para este caso:

MASTER PROG



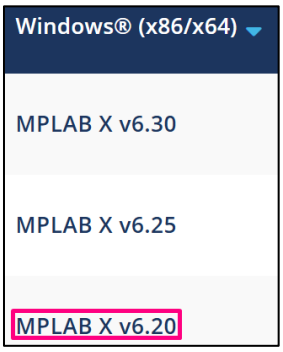
PICKIT 3



Característica	Master Prog	PICkit 3
Tipo de conexión	Cuenta con un **zócalo ZIF de 40 pines** para insertar el chip directamente.	Se conecta a través de un cable **ICSP** (In-Circuit Serial Programming) a una tarjeta externa.
Depuración (*Debugging*)	No soporta depuración avanzada en tiempo real dentro de entornos como MPLAB de la misma forma nativa y robusta.	Diseñado oficialmente por Microchip como **depurador y programador** (*debugger*) para rastrear código paso a paso.
Compatibilidad de software	Utiliza su propio software específico y sencillo para cargar archivos `.hex`.	Integrado de forma nativa con las suites oficiales **MPLAB X** e **MPLAB IDE** de Microchip.
Versatilidad de encapsulados	Ideal para chips en formato **DIP** (gracias a su palanca ZIF).	Requiere adaptar o diseñar pines ICSP en tu placa, permitiendo programar encapsulados pequeños (SMD/SOIC/TQFP) sin desoldar.
Función AutoProg	Destaca por permitir programar de forma automática al momento de compilar el código.	Requiere el flujo de compilación y descarga desde el IDE oficial.

En este caso se instalo una versión de “MPLAB X IDE” especifica para que funcionara el “PICKIT 3” en este caso es la versión 6.20.

https://www.microchip.com/en-us/tools-resources/archives/mplab-ecosystem



Posterior a ello descargamos el compilador: **MPLAB XC8 v4.00**

https://www.microchip.com/en-us/tools-resources/develop/mplab-xc-compilers/xc8



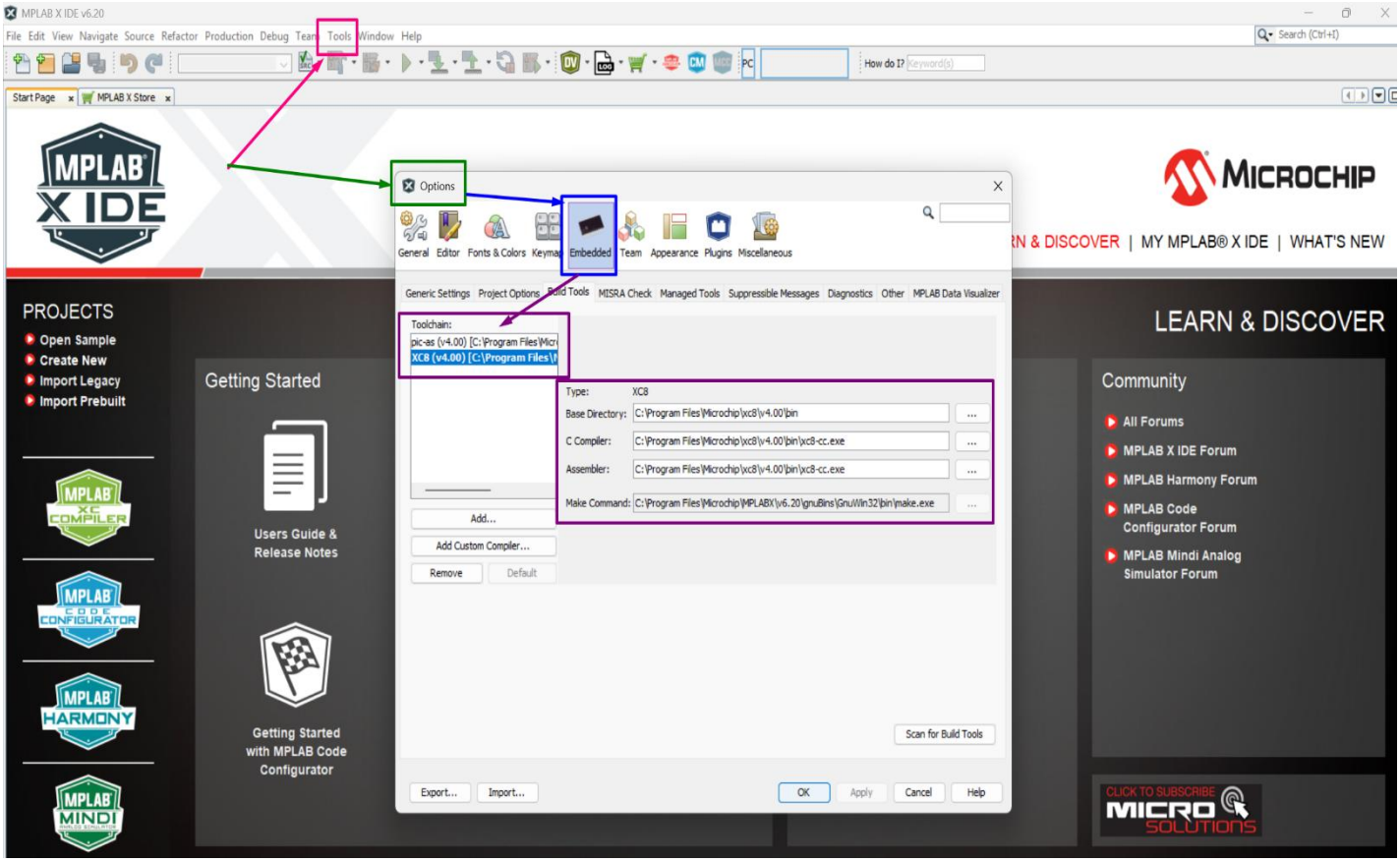
MPLAB X IDE (Entorno de Desarrollo Integrado)

- **Función:** Sirve para el **desarrollo del software**. Aquí escribes el código fuente (en C, C++ o ensamblador), gestionas proyectos y usas herramientas de depuración (*debugging*).
- **Características:** Es un programa pesado y altamente configurable. Incluye editores de código, configuradores gráficos como el *MPLAB Code Configurator* (MCC) y se integra con compiladores como XC8, XC16 o XC32. LinkedIn +3
- **Usuario ideal:** Ingenieros y programadores que están creando y probando la lógica de un programa.

MPLAB IPE (Entorno de Programación Integrado)

- **Función:** Sirve exclusivamente para **flashear o grabar** el microcontrolador. No permite escribir ni modificar código fuente.
- **Características:** Es una aplicación más pequeña, ágil y rápida que se instala automáticamente junto con el IDE. Trabaja a partir de un archivo ya compilado (generalmente un archivo `.hex`). Ofrece opciones avanzadas para entornos de **producción en masa** (como control de números de serie, protección de código y contraseñas de seguridad).
- **Usuario ideal:** Operarios en líneas de fabricación o técnicos que solo necesitan transferir el firmware terminado a los chips. Microchip Technology +4

Para verificar las instalaciones realizadas:



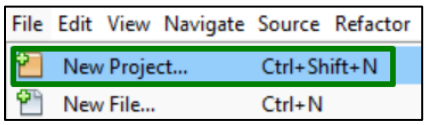
pic-as (v4.00) ☑ ← para Assembly

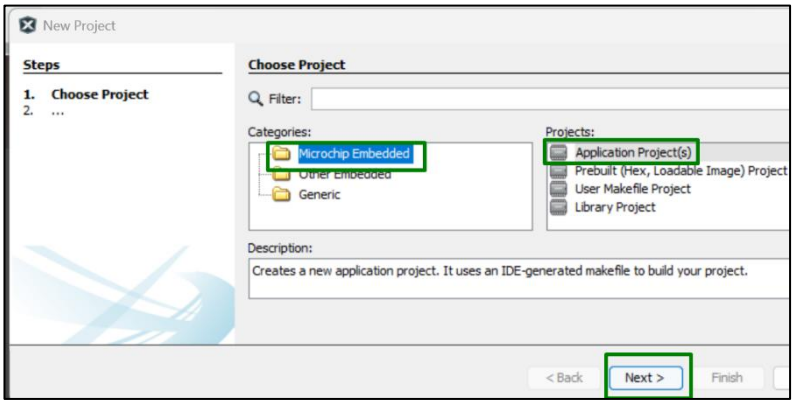
XC8 (v4.00) ☑ ← principalmente para C

Lo siguiente es verificar el reconomiento del “PICKIT3”.

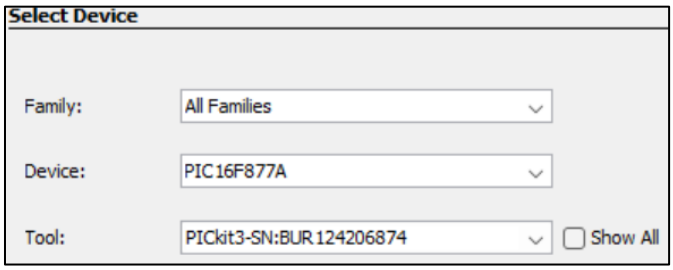


Para ello se crea un proyecto nuevo.



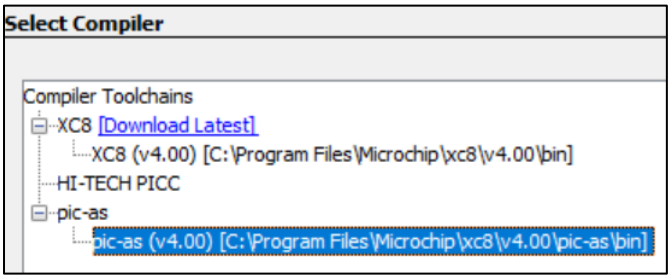


Una vez detectado el dispositivo aparecerá lo siguiente al seleccionar el “MCU”:



`BUR124206874` es el **número de serie de tu PICkit 3**

Selección del compilador



XC8 (v4.00) → para programar principalmente en C

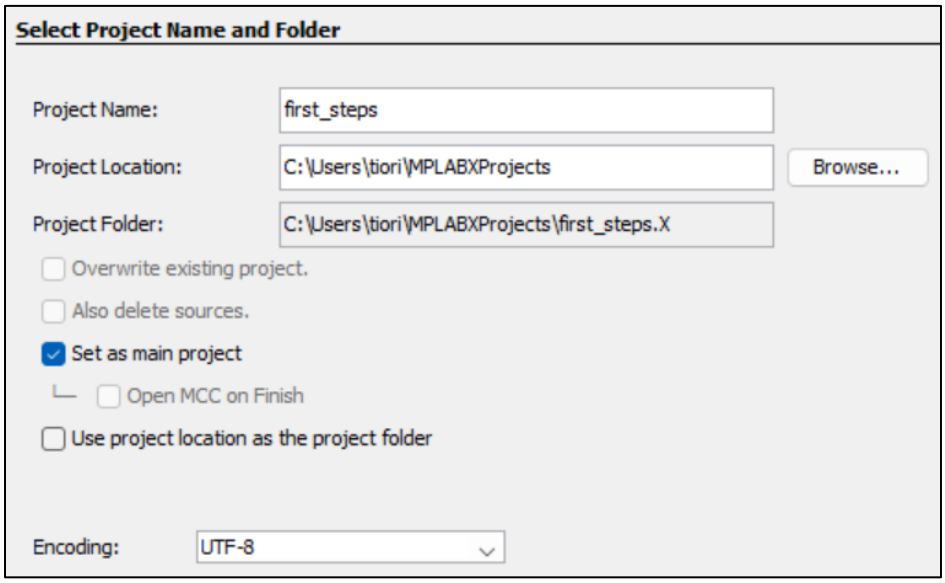
pic-as (v4.00) → para programar en Assembly

En la sección de “encoding” dejar como UTF-8 debido a que habran problemas al momento de generar comentarios traduciendo algo como:

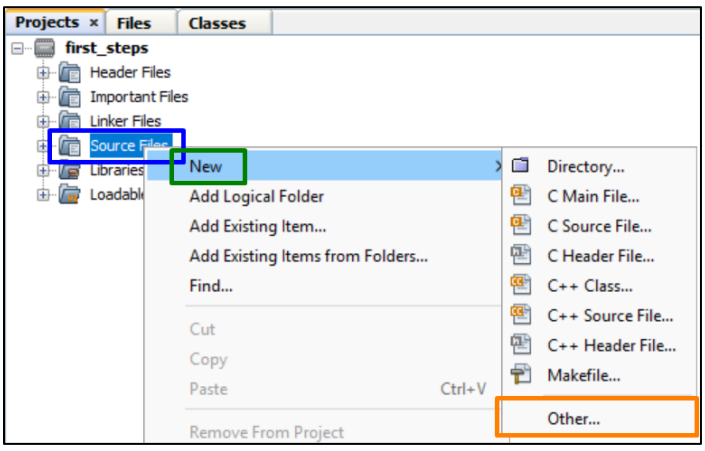
// Configuración del módulo analógico

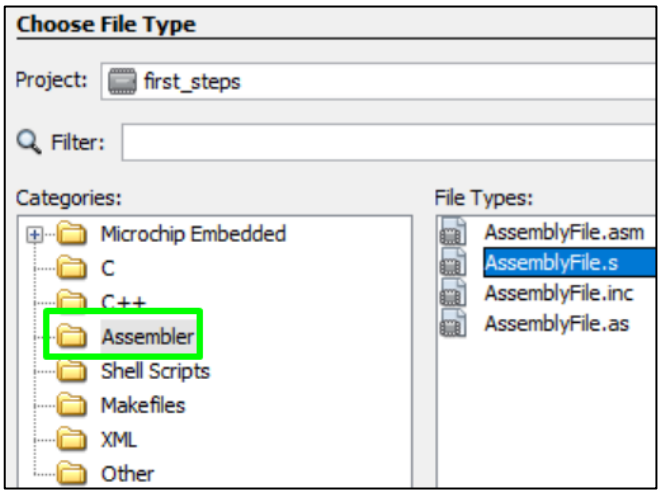
a

ConfiguraciÃ³n del mÃ³dulo analÃ³gico



Creamos un nuevo archivo de fuente:





main.s → Assembly para pic-as

main.S → Assembly para pic-as + preprocesador

main.asm → muy común en proyectos antiguos con MPASM

main.inc → archivo auxiliar de inclusión

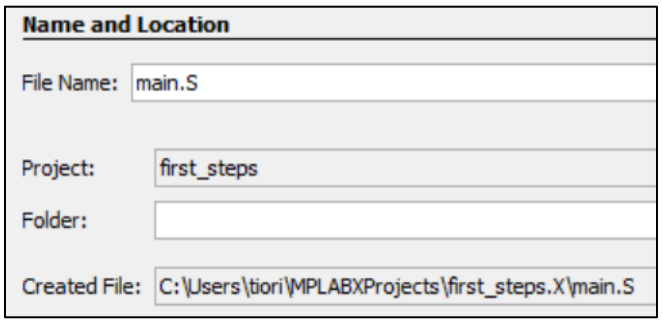
Pero para este propósito vamos a utilizar una versión alternativa para poder generar un archivo con .S mayuscula para que el proceso pase por el “preprocesador”.

main.s → Assembly directo al ensamblador

main.S → preprocesador → Assembly → ensamblador

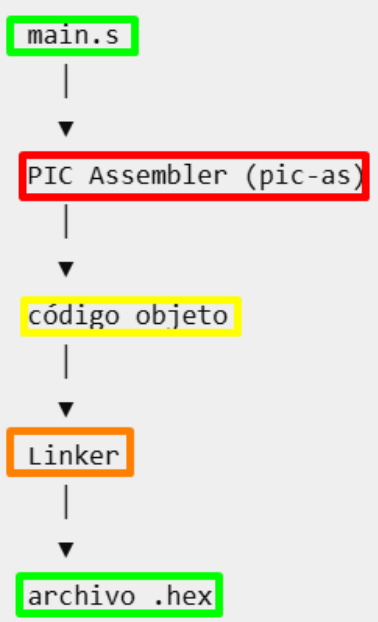
Esto permitiría utilizar “macros” como:

PIC (PORTB, TRISB, STATUS, etc.), macros como BANKSEL

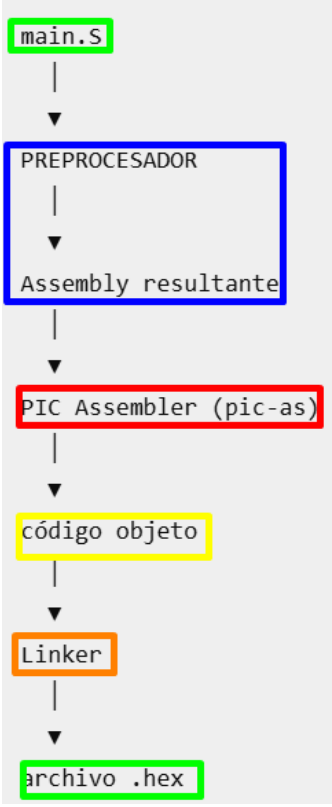


Dato adicional:

Archivo. s (minúscula):

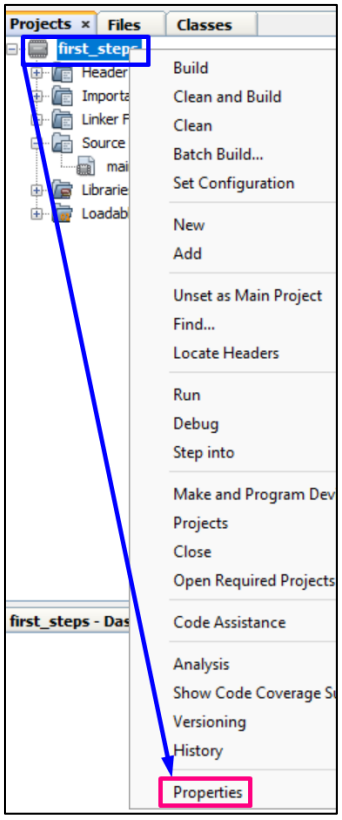


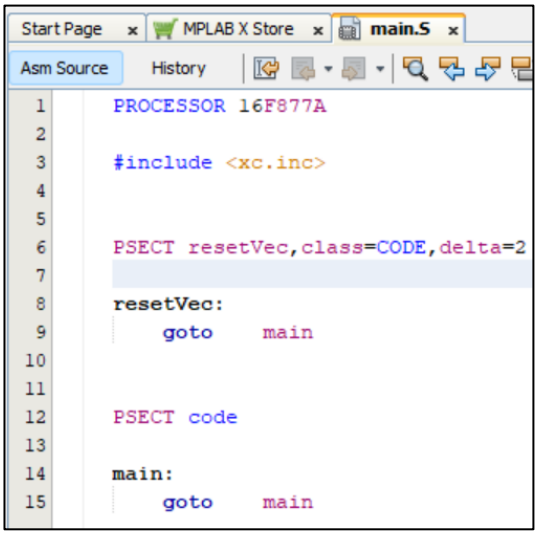
Archivo. S (Mayuscula):



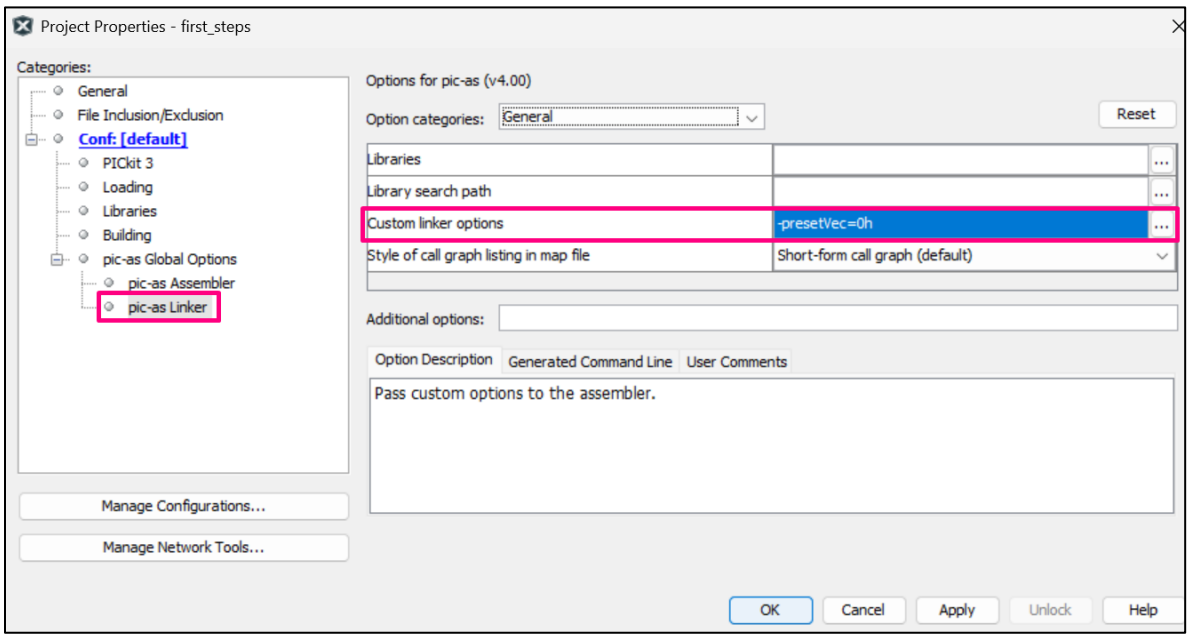
Antes de grabar en el “PIC” físico necesitamos comprobar la funcionalidad del “ensamblado” y la parte del “linker” es decir que el proyecto genere en su flujo correctamente el archivo. hex.

Escribimos el siguiente código de prueba:

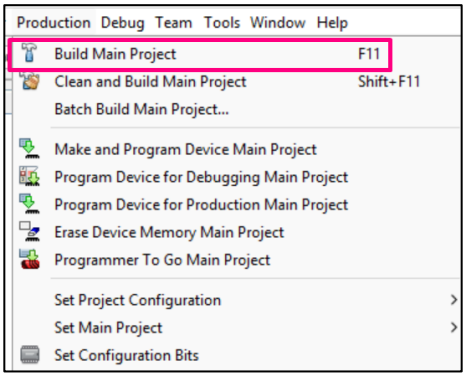


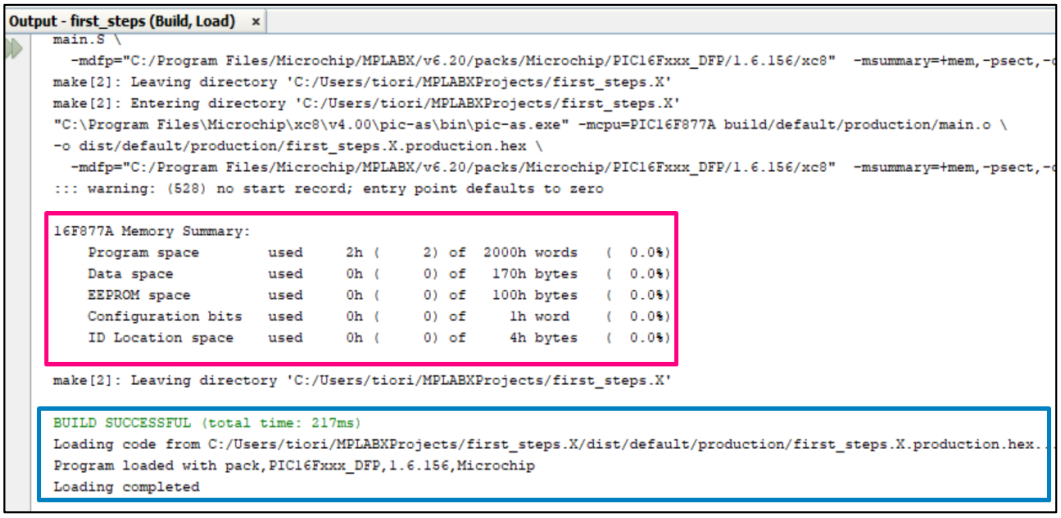


Escribimos en “custom linker”: -presetVec=0h



Guardar el archivo del programa “save” y “build”.





Esto indicaría que el. hex fue generado con éxito.

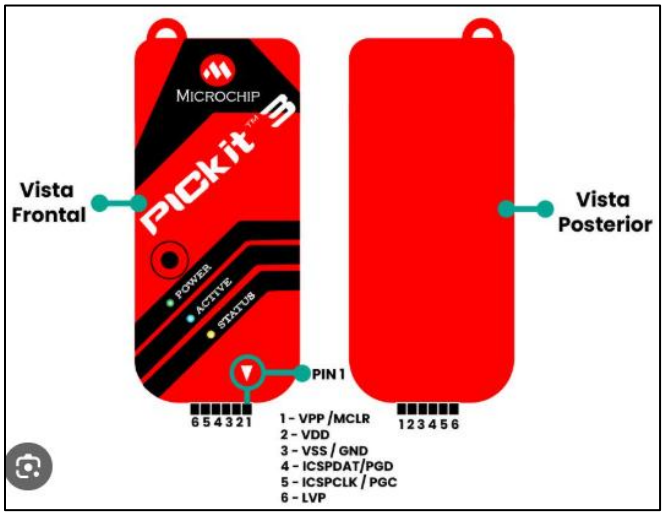
1. **Código fuente (.asm o .s):** Es el archivo de texto donde escribes las instrucciones en ensamblador.

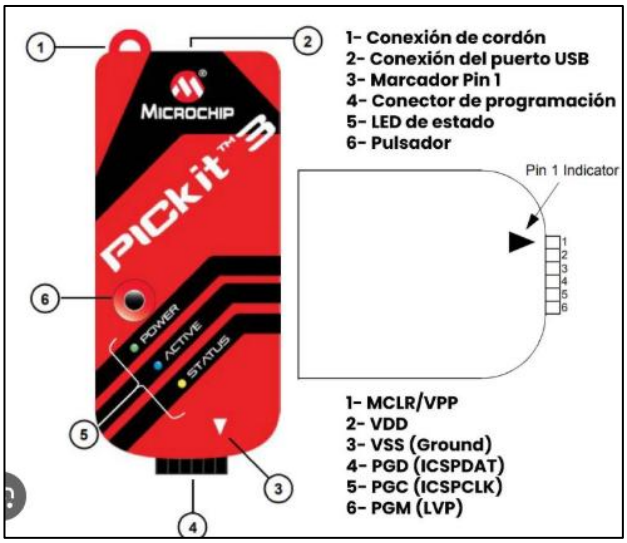
2. **Objeto (.obj o .o):** El compilador/ensamblador traduce tu texto a código máquina intermedio.

3. **Archivo Intel HEX (.hex):** El enlazador (linker) traduce el código objeto a un formato de texto hexadecimal. Contiene las instrucciones máquina junto con las direcciones de memoria exactas donde debe guardarse cada dato y códigos de verificación (checksum).

4. **Archivo Binario (.bin):** Es la imagen binaria pura y directa (unos y ceros) que se copia textualmente bit a bit dentro de la memoria del microcontrolador o chip. No tiene cabeceras ni direcciones de memoria explícitas.

Lo siguiente consistirá en conectar el “PICKIT3”:





Pin PICkit 3	Señal	Función
1	**MCLR/VPP**	Reset y voltaje de programación
2	**VDD Target**	Alimentación/referencia del PIC
3	**VSS**	Tierra (GND)
4	**PGD / ICSPDAT**	Datos de programación
5	**PGC / ICSPCLK**	Reloj de programación
6	**PGM / LVP**	Programación a bajo voltaje

CONSIDERACIONES ESPECIALES CONEXIÓN ENTRE:

"PIC16F877A" – "PICKIT3" EN LA PROTOBOARD

Además de considerar las conexiones dadas por el **"pickit3" al "pic"** deben tomarse en cuenta consideraciones adicionales para elementos que se conectaran.

Una consideración adicional importante es conectar un "capacitor de 100 nF entre

"Vdd" y "Vss" de manera que:

Ese capacitor de **100 nF entre VDD y VSS** funciona como **capacitor de desacoplo o bypass**. Su trabajo es ayudar a que la alimentación del PIC se mantenga estable frente a cambios muy rápidos de corriente.

Aunque tú alimentes el PIC con 5 V, internamente el microcontrolador está conmutando transistores constantemente. Cada vez que cambia el estado de una salida, conmuta lógica interna o ejecuta instrucciones, puede demandar pequeños picos de corriente muy rápidos. El capacitor actúa como una pequeña reserva local:

VDD ——||—— VSS

100 nF

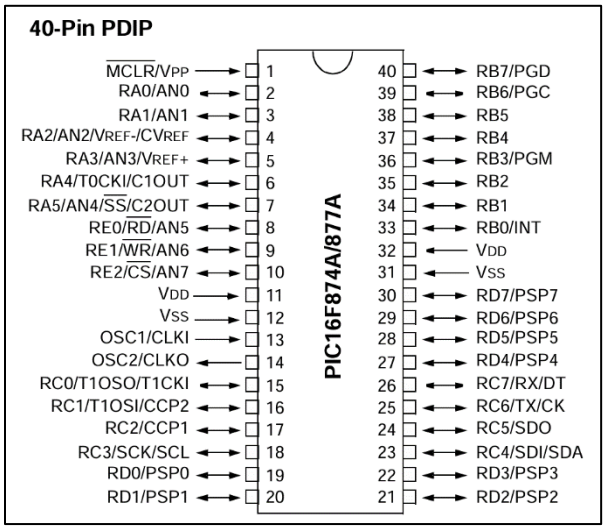
Si el "capacitor" no se conectara podría ocurrir lo siguiente:

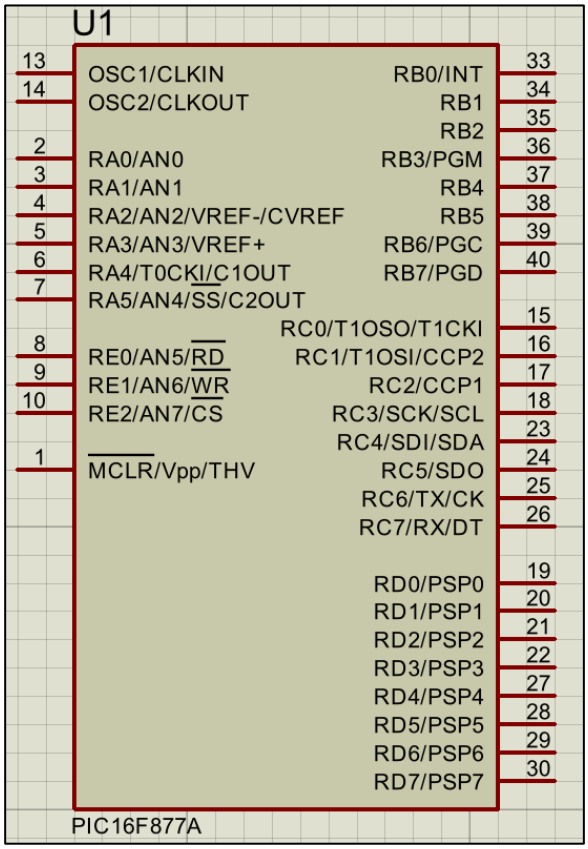
Si **no lo pusieras**, muchas veces el PIC podría funcionar de todas formas, especialmente en una práctica muy sencilla. Pero aumenta la posibilidad de comportamientos inestables, por ejemplo:

- resets aparentemente aleatorios,
- errores al programar o comunicarse,
- lecturas ADC inestables,
- fallos cuando cambian varias salidas,
- problemas al conectar cargas,
- funcionamiento correcto unas veces y otras no.

Esquematico conexiones “Pic16f877a” a “pickit3” – Proteus 9.2 Demo Version (Landscape 4)

Algo importante a considerar aquí es que el “pic16f877a” físico y proporcionado por microchip aparece idéntico, pero en “proteus” esta organizado en base a los grupos funcionales.





- **MCLR (Master Clear):** permite reiniciar el microcontrolador mediante una señal activa en nivel bajo.

- **VPP (Programming Voltage):** recibe la tensión de programación cuando utilizamos el PICkit 3.

Se debe colocar una resistencia de pull-up de 10 kΩ entre el pin MCLR (Pin 1) y VDD (5V) del microcontrolador PIC16F877A

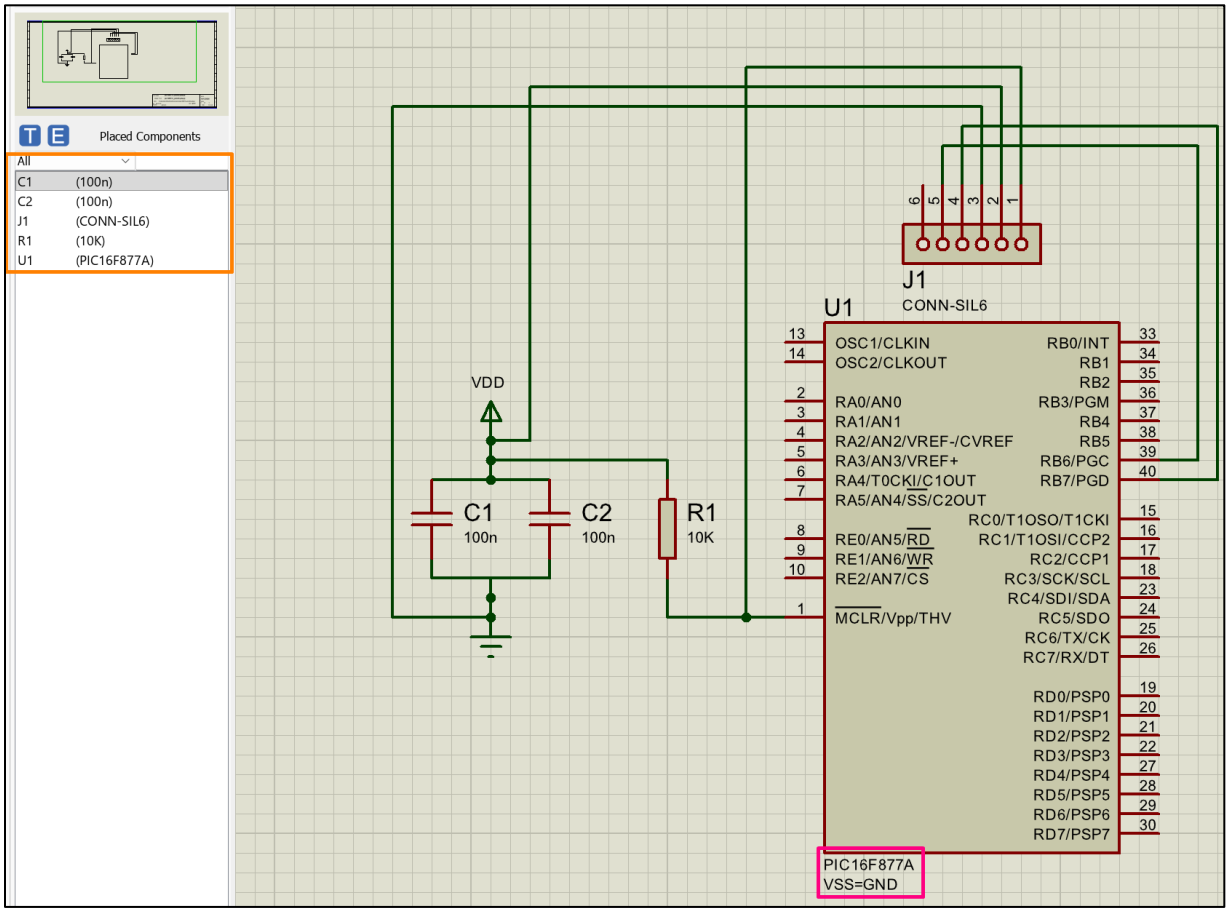
¿Por qué es necesaria?

- **Estado lógico alto:** El pin MCLR es el *Master Clear* (reinicio) y funciona con lógica invertida (activo en bajo). Debe recibir un **1 lógico (5V)** de forma permanente para que el PIC funcione con normalidad. Microcontrollers Lab +2

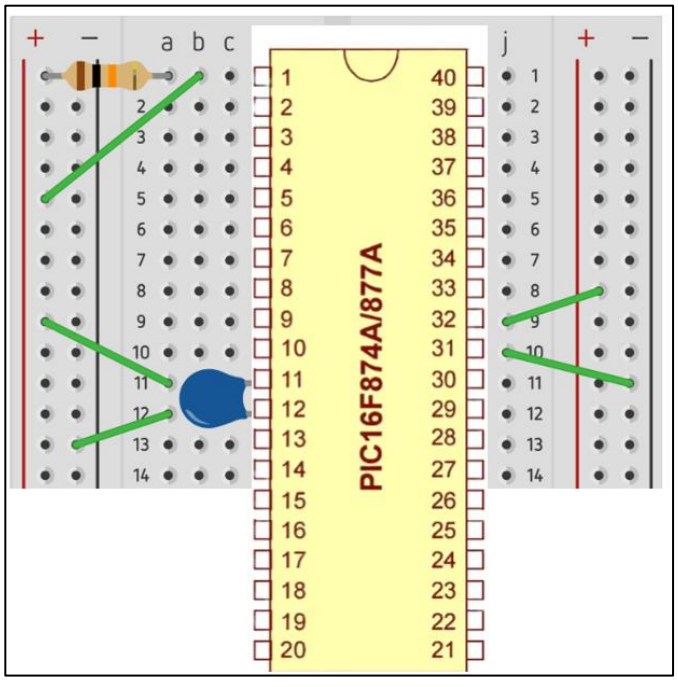
- **Evita reinicios aleatorios:** Si el pin se deja al aire (flotante), el ruido eléctrico puede enviar un pulso de 0V y hacer que el microcontrolador se reinicie constantemente. Microcontrollers Lab +1

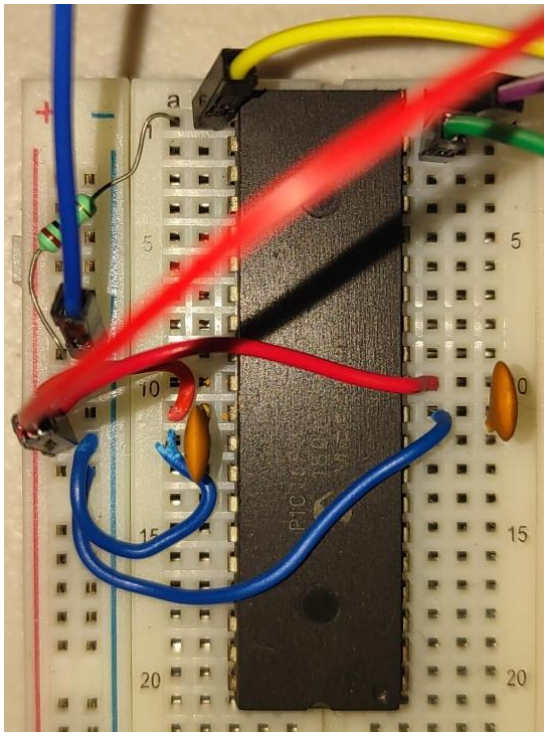
- **Protección al usar un botón de reset:** Si decides agregar un botón (*push-button*) conectado entre MCLR y GND para reiniciar el PIC manualmente, la resistencia de 10 kΩ evita que se produzca un cortocircuito directo entre VDD y GND al presionar el botón. MIKROE +1

Pin 3	**VSS/GND**	Pines 12 y 31
Pin 4	**PGD (ICSPDAT)**	Pin 40 (RB7)
Pin 5	**PGC (ICSPCLK)**	Pin 39 (RB6)
Pin 6	**PGM/AUX**	No conectar inicialmente



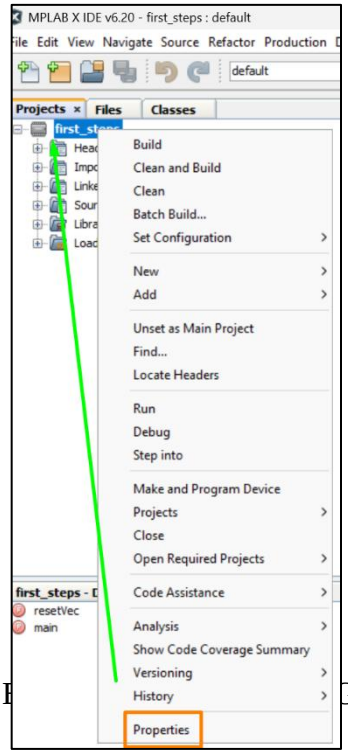
IMÁGENES DE EJEMPLO

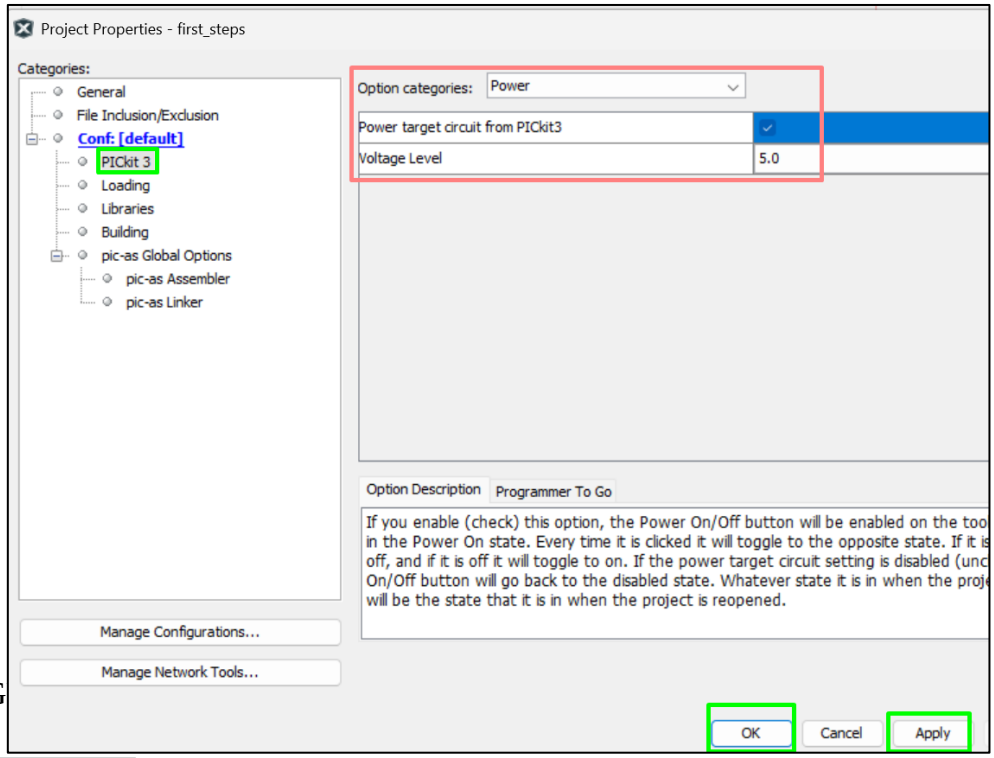


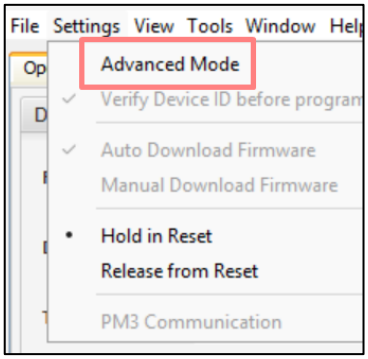


NOTA: También colocar una resistencia de 10kohms en el pin36 conectada a GND.

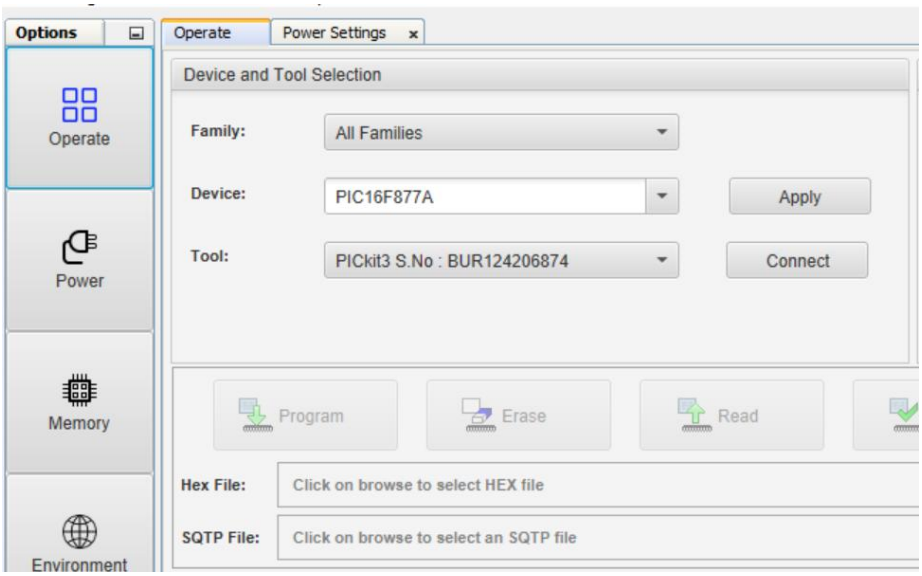
Lo siguiente consistirá en activar “PICKIT3 – para alimentación”:





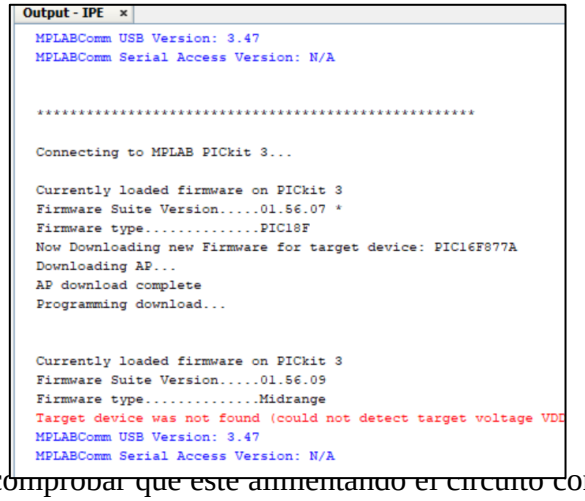






Marcar casilla, desconectar y volver a conectar nuevamente y en la terminal marcara algunos errores, pero en el multímetro debe dar 5 volts aproximadamente entre VDD Y VSS del PICKIT3.





Realizamos mediciones con el multímetro para comprobar que este alimentando el circuito con 5 volts la cual debería oscilar:

4.86 volts hacia adelante.

Si al hacer click en “Connect” aparece el siguiente error:

1. ¿Qué significa exactamente el código 0x3FE0?

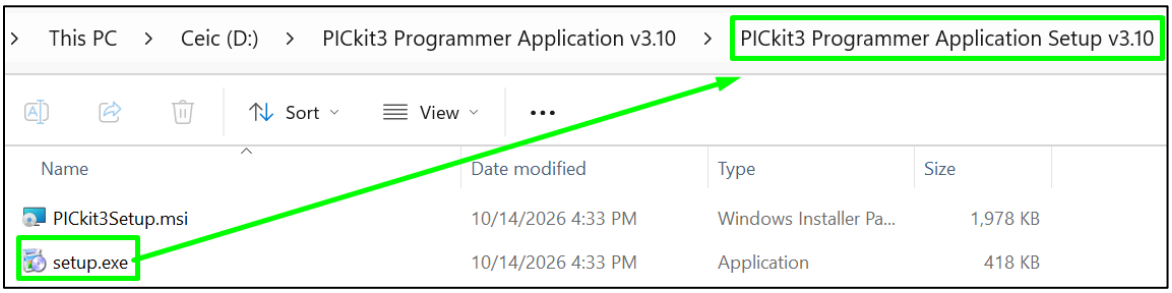
El PIC16F877A tiene un identificador que permite al programador reconocerlo.

Concepto	Valor
Microcontrolador	PIC16F877A
Device ID esperado	`0x0E20` (con bits de revisión)
Device ID obtenido	`0x3FE0`
Resultado	Identificación incorrecta

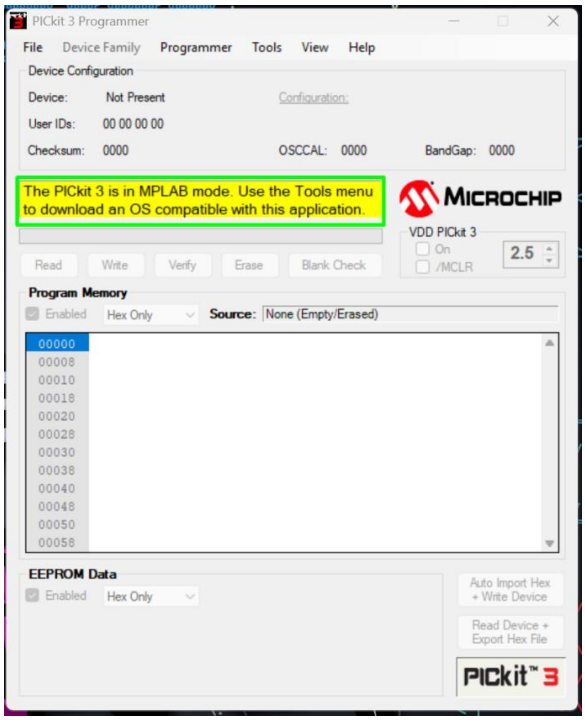
Un paso extra a considerar para poder reconocer el ID device es descargar el ejecutable del programador Pickit3:

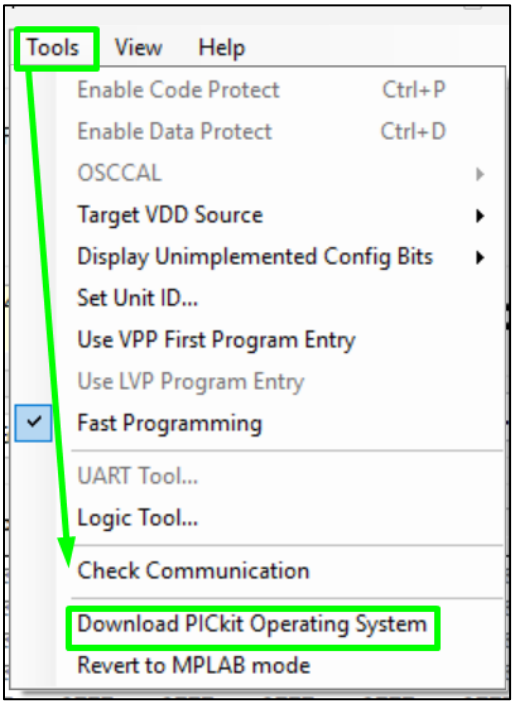
https://www.microchip.com/en-us/tools-resources/archives/mplab-ecosystem

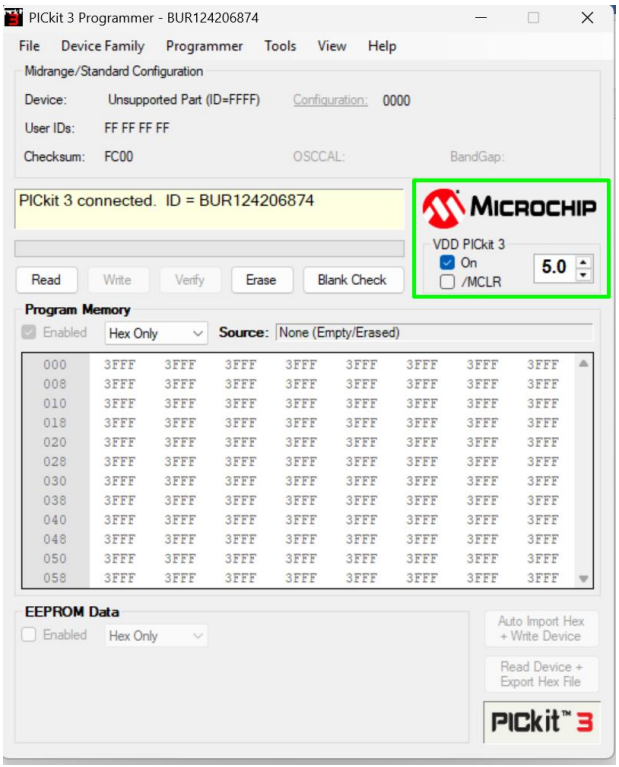




Es necesario cambiarlo a **“scripting mode”:**

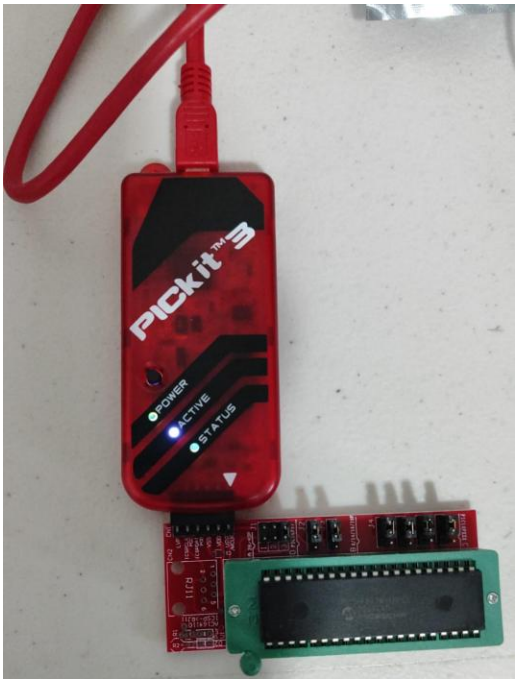






Una vez detectado el pic tendremos algo como lo siguiente en MPLAB IPE:

En este caso particular se utilizo la placa con el zocalo ZIF para una detección mas inmediata, el circuito anterior tambien puede proveer del mismo funcionamiento.




```
**********************************************************

Connecting to MPLAB PICkit 3...

Currently loaded firmware on PICkit 3
Firmware Suite Version.....01.56.09
Firmware type..............Midrange
Programmer to target power is enabled - VDD = 5.000000 volts.
Target device PIC16F877A found.
Device Revision ID = 8
```

MPLAB X IDE 6.20 → escribir código, compilar, depurar

pic-as v4.00 → ensamblador

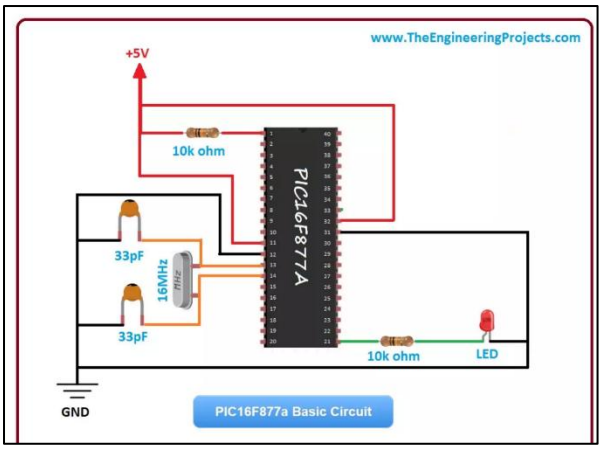
PIC16F877A → microcontrolador que vamos a estudiar

PICkit 3 nuevo → programar/depurar el PIC

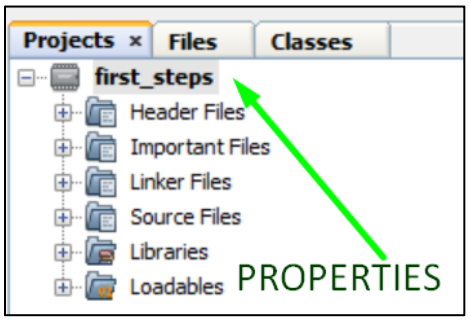
Adaptador ZIF → conexión física cómoda

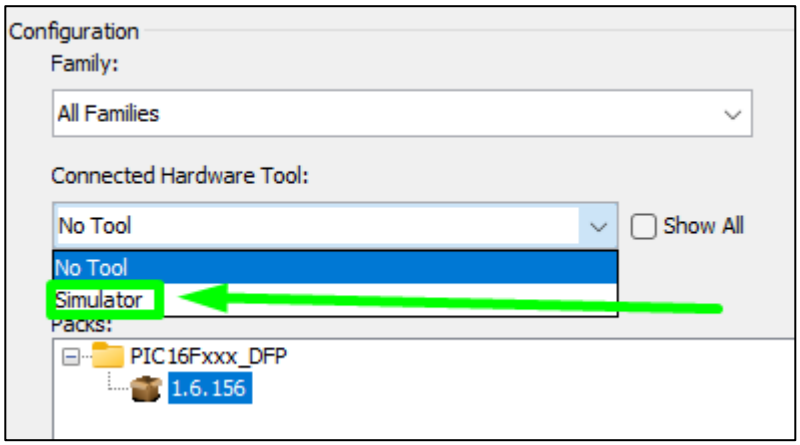
MPLAB Simulator → estudiar internamente la CPU sin hardware

PIC16F877A es un microcontrolador RISC de 8 bits**, con arquitectura** Harvard**.**



PRUEBA DE SIMULACIÓN EN **MPLAB X IDE:**





Dato adicional importante:

El preprocesador es una herramienta de software que actúa antes de la compilación, mientras que el procesador (o microcontrolador) es el componente de hardware que ejecuta el código final.

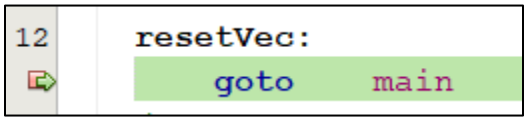
Característica	Preprocesador	Procesador (MCU)
¿Qué es?	Un **programa de software** (parte del compilador/ensamblador).	El **chip físico** (hardware) que ejecuta las instrucciones.
¿Cuándo actúa?	**Antes** de que el código se convierta en lenguaje de máquina (tiempo de diseño).	**Durante** la ejecución del circuito (tiempo de ejecución).
¿Qué procesa?	Texto, directivas y macros (ej. `#define`, `INCLUDE`).	Instrucciones binarias en lenguaje de máquina (ej. `MOVLW`, `ADDWF`).
Consumo de memoria	No ocupa memoria en el chip; solo ayuda a organizar el código fuente.	Consume la memoria Flash y RAM del microcontrolador.

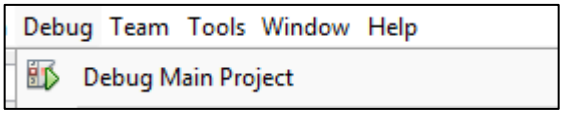
Seteamos un **"breakpoint"** para observar el comportamiento:

- **Build for Debugging Main Project**: compila el proyecto con información de depuración, pero no necesariamente inicia la ejecución del debugger.

- **Clean and Build for Debugging Main Project**: borra primero los archivos compilados anteriores y luego recompila todo para debug. Es útil si sospechamos que quedó algo viejo o extraño en la compilación.

- **Debug Main Project**: compila si hace falta y además inicia la sesión de depuración. **Este es el que debes usar ahora.**





Lo siguiente es acceder al **SFRs.**

Window → Target Memory Views → SFRs

Se abrirá una ventana con los **Special Function Registers** del PIC16F877A.

En esta parte se observa un comportamiento interesante en la parte de PCL

Address	Name	Hex	Decimal	Binary
	WREG	0x00	0	00000000
000	INDF	0x00	0	00000000
001	TMR0	0x00	0	00000000
002	PCL	0x00	0	00000000
003	STATUS	0x18	24	00011000

Al presionar **F7** posteriormente deberíamos visualizar:

Step Into -- F7 nos permite observar el microcontrolador casi como si pudiéramos decirle:

ejecuta solamente una instrucción y espera.

Address	Name	Hex	Decimal	Binary
	WREG	0x00	0	00000000
000	INDF	0x00	0	00000000
001	TMR0	0x00	0	00000000
002	PCL	0xFF	255	11111111
003	STATUS	0x18	24	00011000

PCL – Program counter low – registro de 8 bits.

El PIC16F877A tiene un **Program Counter (PC)** de 13 bits, porque puede direccionar hasta 8192 palabras de memoria de programa:

PC = 13 bits

0000000000000_2 → 0x0000

...

1111111111111_2 → 0x1FFF

Pero PCL solamente nos permite ver/acceder a los **8 bits menos significativos** de ese contador:

Program Counter completo

PC12 PC11 PC10 PC9 PC8 | PC7 PC6 PC5 PC4 PC3 PC2 PC1 PC0

|<--------- PCL --------->|

Al estar en binario **0001 1111 1111 1111 "PC COMPLETO"**

Por lo tanto **1111 1111** PCL – es decir "255".

En **"program Memory" podemos observar como se genero el comportamiento del "linker" al presionar "step into" y entrar a "main – go to main"**

```
23   */
24   resetVec:
25       goto    main
26
27
28
29   /*
30   main
31     ↓
32   GOTO main
33     ↓
34   main
```

Output | Variables | Call Stack | Breakpoints | SFRs | Program Memory

	Line	Address	Opcode	Label	DisAssy
➪	1	0000	2FFF	__S1	GOTO 0x7FF
	2	0001	3FFF		ADDLW 0xFF
	3	0002	3FFF	__Hudata_bank0	ADDLW 0xFF
	4	0003	3FFF		ADDLW 0xFF
	5	0004	3FFF		ADDLW 0xFF
	6	0005	3FFF		ADDLW 0xFF
	7	0006	3FFF		ADDLW 0xFF
	8	0007	3FFF		ADDLW 0xFF
	9	0008	3FFF		ADDLW 0xFF
	10	0009	3FFF		ADDLW 0xFF

	2047	07FE	3FFF		ADDLW 0xFF
➪	2048	07FF	2FFF		GOTO 0x7FF
	2049	0800	3FFF	__Hspace_0	ADDLW 0xFF

El siguiente comportamiento en el **linker** se interpretaría asi:

